

Building a Computer From Scratch

From a first-year question about logic gates to a working compiler

Eduard Hayrapetyan

Contents

Contents	1
1. The question that started it all.....	2
2. The big idea: abstraction.....	2
3. Getting gates: silicon, transistors, and the first logic.....	3
4. From gates to math: adders and the ALU	6
5. Teaching the machine to remember	9
6. Giving the machine a language: instruction set design	12
7. Wiring it all together: microarchitecture	14
8. Tools so I don't have to hand-encode binary.....	17
9. The last mile: designing programming language and compiler	18
10. Closing: answering first-year me.....	23
References	24

1. The question that started it all

First year of college, somewhere in a discrete math or intro CS course, I learned Boolean algebra: AND, OR, NOT, truth tables, De Morgan's laws. At some point a question occurred to me: if I had some gates, I could build math out of them - an adder, maybe more than that. It felt like glimpsing the edge of something big. But right behind that came a more basic question that the course never really answered: where do the gates actually come from? Most textbooks hands you AND and OR as if they arrived fully formed, ready to be wired together, without showing the floor underneath them. That question stayed with me. I'm not a hardware engineer nor am i a compiler engineer - just someone who wanted an answer. This article (and the thesis behind it) is the answer I eventually put together: a chain from transistors that don't exist yet to a program in a language I designed, compiling and running on a processor I built. Please note that this is a simplified model, meant to give a general understanding of what is going on. Here's where it ends up, for a small taste of the destination:

```
func add(a, b) { return a + b; }
func main() {
    var x = add(10, 20);
    var y = add(x, 5);
    return y;
}
```

This compiles, assembles, and runs hardware described, gate by gate, starting in the very next section.

2. The big idea: abstraction

A modern processor packs roughly a billion transistors into a single square centimeter. Nobody - not even the engineers who designed it - reasons about that chip one electron at a time. The only reason any of this is buildable, or understandable, by a human brain is **abstraction**: each layer hides the details of the layer below it and exposes a simpler interface above. A computer can be split into nine such layers:

1. Physics	electrons, conductivity, semiconductor behavior
2. Devices	transistors and diodes
3. Analog circuits	amplifiers and filters
4. Digital circuits	logic gates that treat voltage as 0s and 1s
5. Logic circuits	gates combined into arithmetic and memory
6. Microarchitecture	how a processor physically executes instructions
7. Architecture	the instruction set itself
8. Operating system	drivers, kernel, resource management
9. Software	the programs people actually use

This project lives mostly in layers 4 through 7, with a quick stop at layers 1 and 2 to earn the right to use a logic gate. From there it climbs back up through the tools that turn raw hardware into something programmable: an assembler, an emulator, and finally a small language with its own compiler - silicon, through transistors, logic gates, arithmetic circuits, memory, a complete CPU, an instruction set, an assembler and emulator, a custom language, and a compiler.

3. Getting gates: silicon, transistors, and the first logic

Why silicon

Pure silicon is barely a conductor - closer to an insulator. Each atom has four valence electrons, forming a rigid crystal lattice. The trick that makes silicon useful is **doping**: introducing tiny amounts of impurities into that lattice to deliberately change how well it conducts.

- Dope it with a Group V element like arsenic, and you get extra free electrons - n-type silicon.
- Dope it with a Group III element like boron, and you get “holes” - missing electrons that act like positive carriers - p-type silicon.

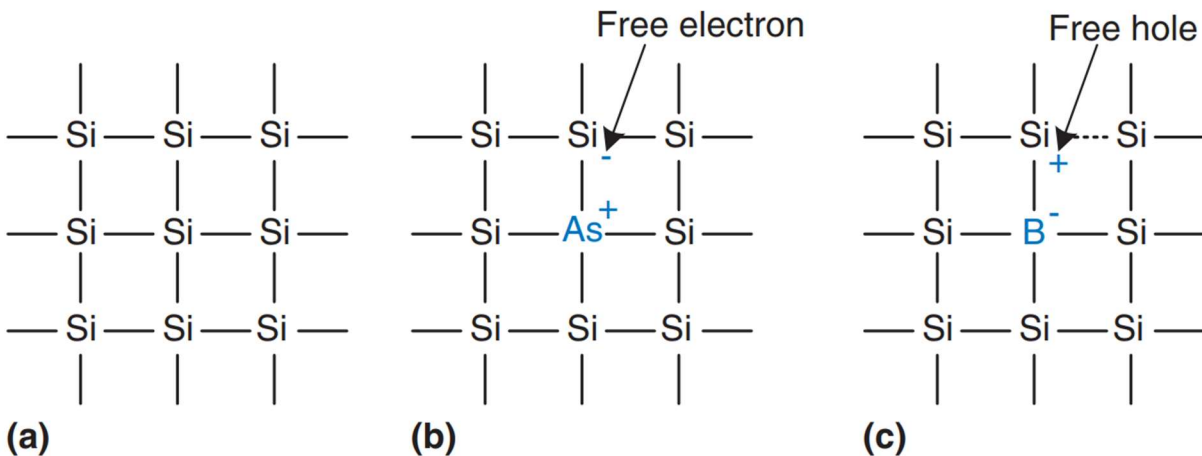


Figure 1: Silicon doping – (a) pure lattice (b) n-type (arsenic) (c) p-type (boron)

Diodes

Put p-type and n-type silicon next to each other and the result is a **diode** - a one-way valve. Current flows from the p-side (anode) to the n-side (cathode) once the voltage crosses roughly 0.7V, and is blocked the other direction.

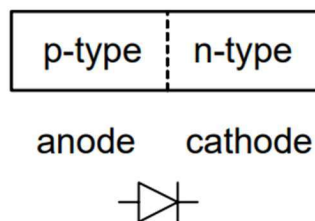


Figure 2: Diode structure and schematic symbol

The MOSFET

The real building block of every digital system is the **MOSFET** - a voltage-controlled switch. A conducting gate sits on an insulating layer of silicon dioxide, which sits on the silicon substrate. Voltage on the gate creates an electric field that opens or closes a channel between source and drain.

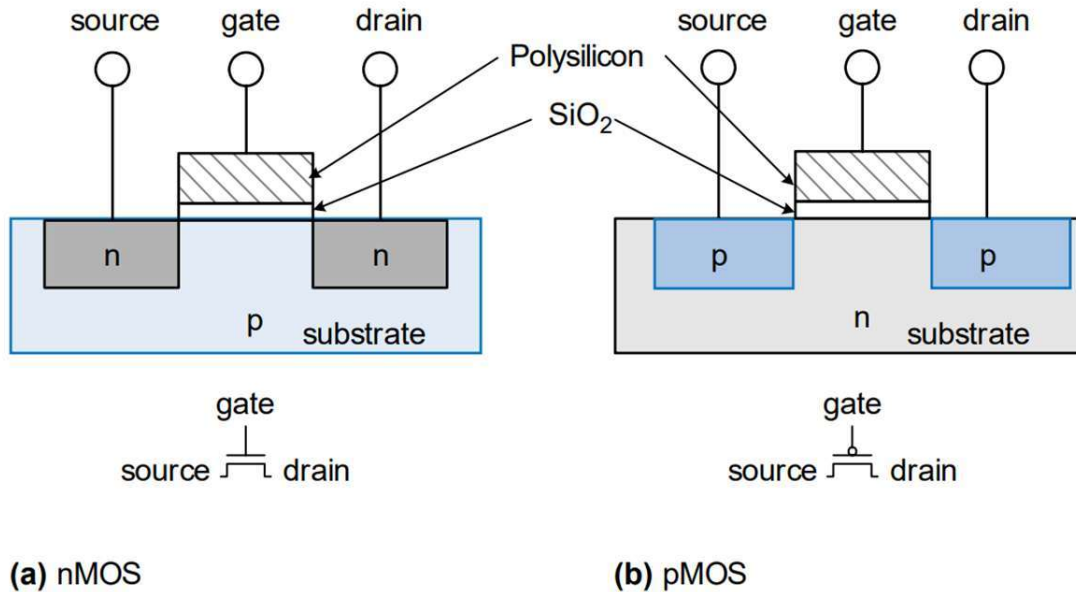


Figure 3: MOSFET structure - nMOS and pMOS

There are two flavors. **nMOS** turns on when the gate voltage is higher than the source by a threshold. **pMOS** turns on when the gate voltage is lower than the source by a threshold. That's the entire trick. Wire nMOS and pMOS transistors together in the right pattern, and logic gates fall out.

The clock

Before building gates that do something over time, a digital system needs a heartbeat: the **clock signal generator**, typically built around a quartz resonator, producing a steady square wave alternating between high and low voltage.

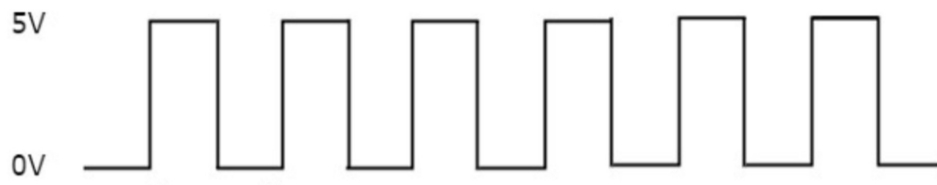


Figure 4: Clock signal: a steady square wave alternating between 0V and 5V

The rising edge (0→1 transition) and falling edge (1→0) matter as much as the levels themselves - they're what later lets a flip-flop know exactly when to update.

NOT - the first gate

The simplest gate, NOT, is just one nMOS and one pMOS transistor sharing an input. When the input is 0, the pMOS conducts and pulls the output to 1. When the input is 1, the nMOS conducts and pulls the output to 0.

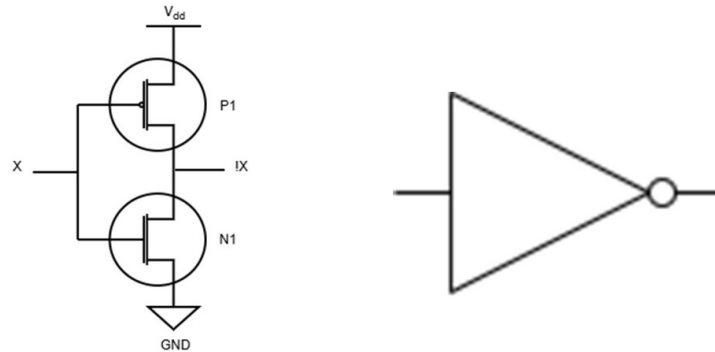


Figure 5: NOT gate - transistor schematic and circuit symbol

NOR and NAND

From there: **NOR** consists of two pMOS in series, two nMOS in parallel and **NAND** consist of two pMOS in parallel, two nMOS in series.

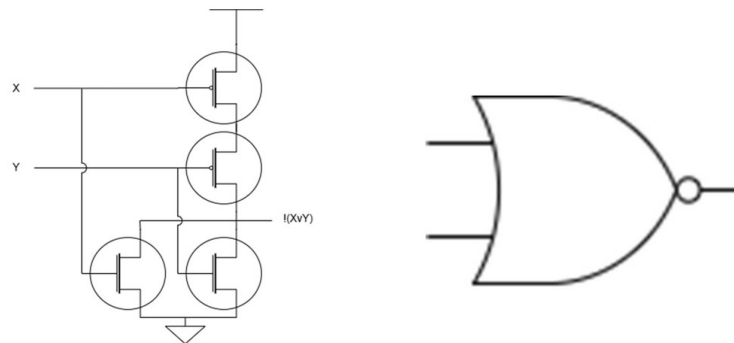


Figure 6: NOR gate - transistor schematic and circuit symbol

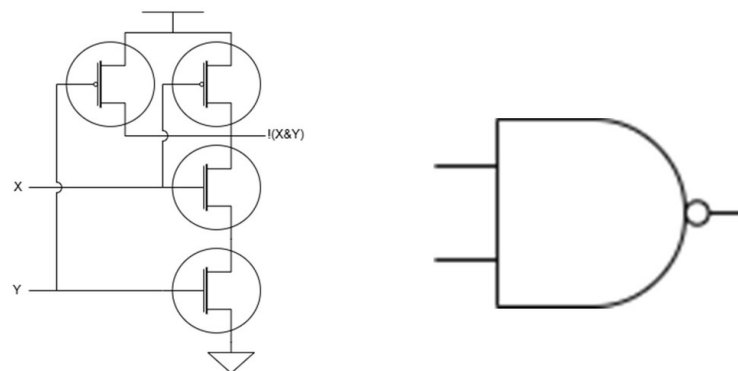


Figure 7: NAND gate - transistor schematic and circuit symbol

Together NOT, NAND, and NOR are **functionally complete** — any Boolean function can be built purely out of any one of them. NAND is also functionally complete, though in practice nobody builds everything from NAND alone; using the full set of gates produces far more readable and efficient circuits. Here are some other gates that are frequently used. Any of those can have more inputs than shown, but logic behind it stays unchanged.

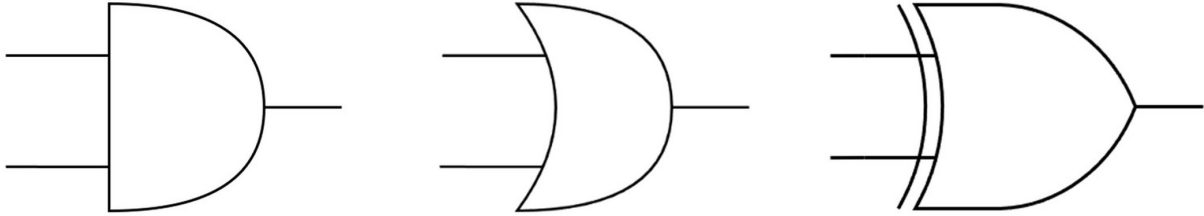


Figure 8: AND, OR, and XOR circuit symbol

4. From gates to math: adders and the ALU

Half adder

Take two bits, x and y , and look at their sum and carry-out. The sum column is exactly XOR's truth table. The carry column is exactly AND's truth table. A **half adder** is nothing more than an XOR gate and an AND gate sharing inputs.

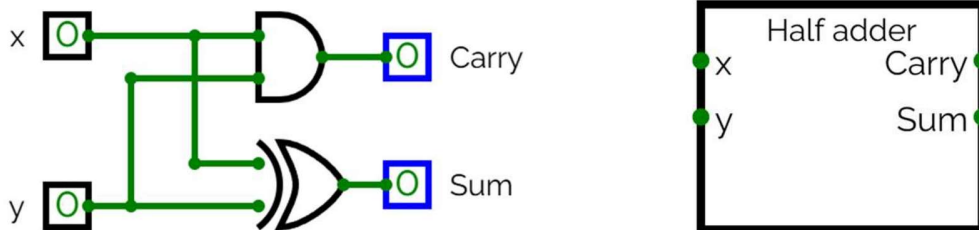


Figure 9: Half adder - circuit and block symbol

Full adder

A half adder can't be chained - it has nowhere to accept a carry **in** from the previous bit. Add a third input, C_{in} , and the result is a **full adder**, built from two half adders with their carries OR'd together.

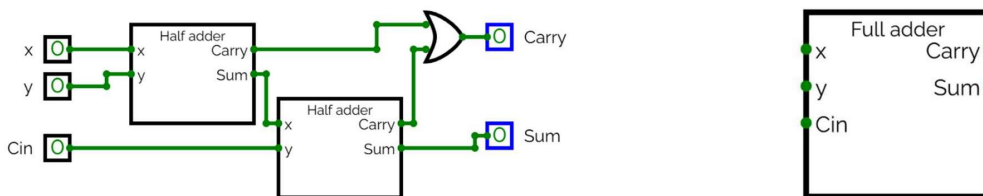


Figure 10: Full adder - circuit and block symbol

Chain full adders together, carry-out feeding the next carry-in, and you can add numbers of any width:

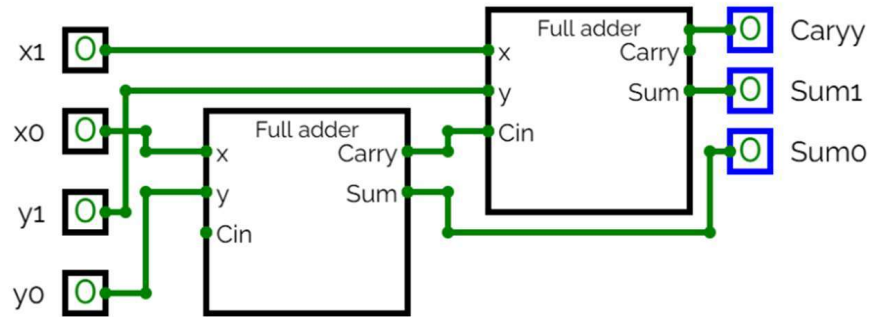


Figure 11: Two-bit addition using chained full adders

That kind of chaining is not the fastest way but it is the simplest.

Subtraction

A full subtractor follows nearly identical logic, with Bin/Borrow standing in for Cin/Carry. In practice, subtraction is often just addition in disguise: feed the adder the inverted second operand with an initial carry-in of 1, and it performs two's-complement subtraction for free.

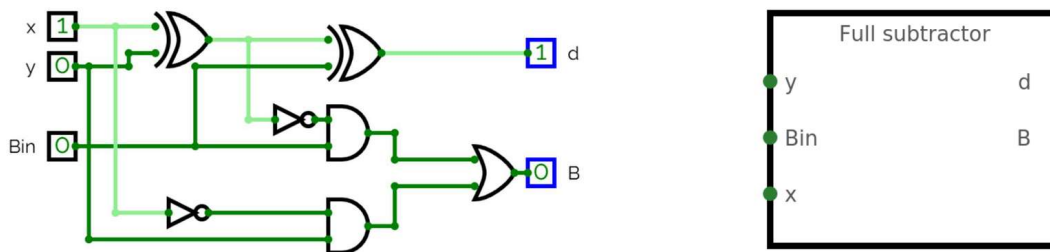


Figure 12: Full subtractor - circuit and block symbol

Deciding which operation to run

A **binary decoder** takes an n-bit code and activates exactly one of 2^n output lines - this is how an opcode field gets turned into "run this specific operation."

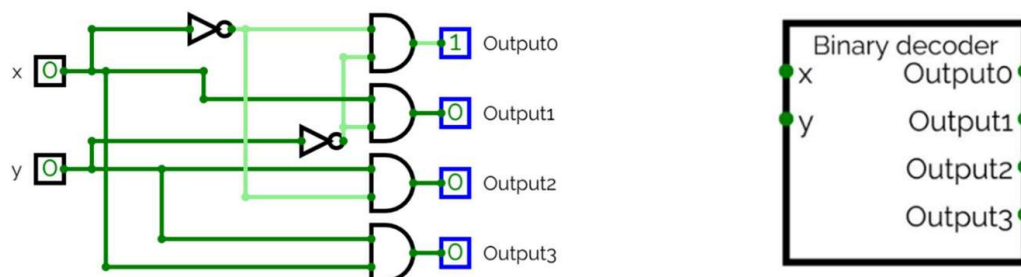


Figure 13: 2:4 binary decoder - circuit and block symbol

A **multiplexer (MUX)** selects one of several inputs based on a control code and routes it to a single output, like a switchboard operator. A **demultiplexer (DEMUX)** does the reverse: one input routed

to one of several possible outputs, like a mail carrier delivering a single letter to exactly the right house.

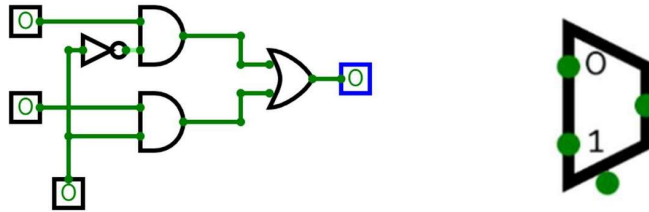


Figure 14: Multiplexer - circuit and symbol

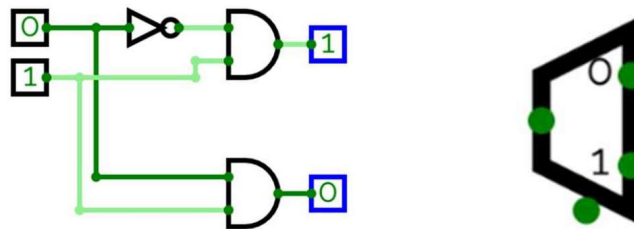


Figure 15: Demultiplexer - circuit and symbol

The ALU

Combine an adder, a subtractor, AND/OR/XOR gates, and a multiplexer to select between their outputs, and the result is an **Arithmetic Logic Unit (ALU)** - the computational core of the processor. The 8-bit ALU used here takes two operands and a 3-bit operation code, and produces a result plus three status flags: **Zero** (result is all 0s), **Sign** (mirrors the result's most significant bit), and **Carry** (set on overflow or borrow).

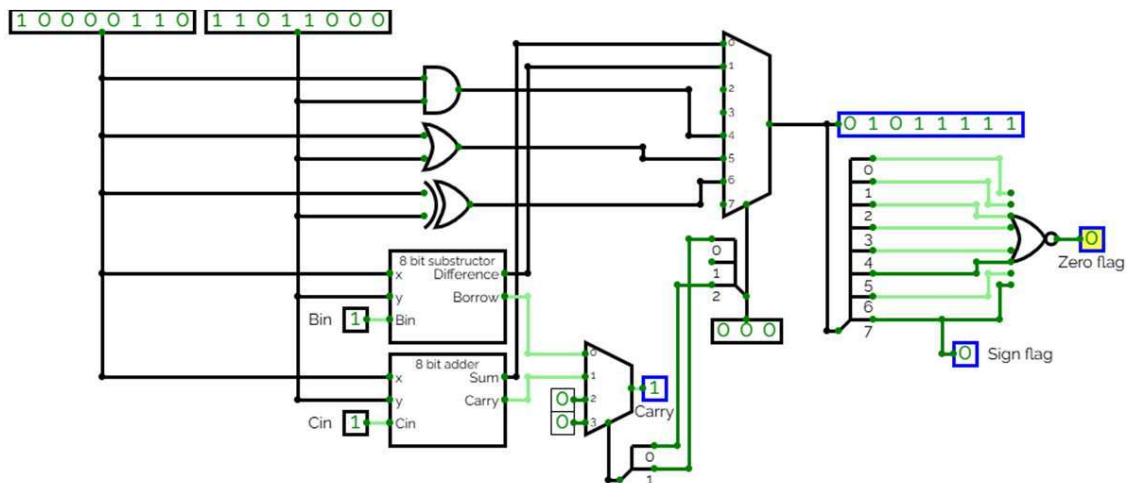


Figure 16: 8-bit ALU circuit

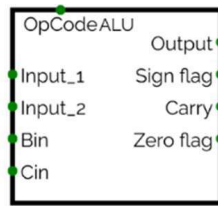


Figure 17: ALU block symbol

Those flags become essential later - they're how **if (a == b)** in a high-level language eventually becomes a hardware decision.

5. Teaching the machine to remember

Everything so far is **combinational** - output depends only on current input, with no concept of "before." A computer needs to remember. Building memory out of components that individually have none turns out to be one of the more satisfying parts of the whole stack.

SR latch

The starting point: two cross-coupled NOR gates with inputs Set and Reset. It has four behaviors - set, reset, an invalid state where both outputs collapse to 0 (when both set and reset are 1), and a "hold" state, where the latch settles back into whatever value it held before. That hold behavior is the first glimmer of memory.

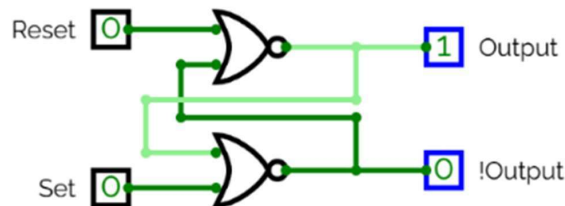


Figure 18: SR latch schematic

D latch

The invalid state is a problem, so the inputs get gated behind a clock signal, producing a **D latch**. When CLK = 0, the data input is ignored and the output holds steady. When CLK = 1, the output directly tracks the input ("transparent" mode).

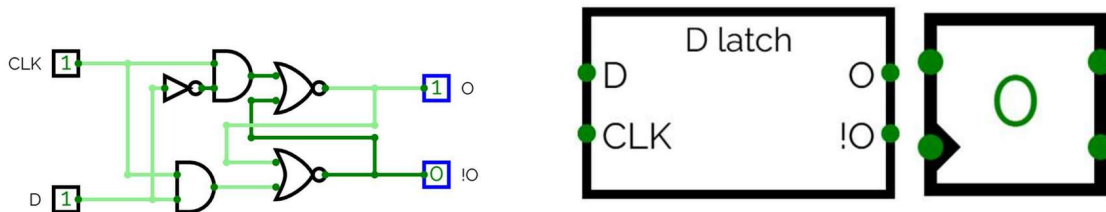


Figure 19: D latch - circuit and block symbol

That transparency turns out to be its own problem: feed a D latch's own inverted output back into its input - exactly the kind of feedback a real processor needs - and while $\text{CLK} = 1$, the circuit oscillates uncontrollably. This single issue is the reason a CPU can't be built directly out of D latches.

D flip-flop

The fix: two D latches in series - Master and Slave - sharing a clock but inverted relative to each other. While $\text{CLK} = 0$, the master captures the input and the slave holds steady, so nothing changes at the output. Only on the clock's rising edge does the captured value propagate through. This edge-triggered behavior is what makes synchronous digital design possible - it cuts continuous time into discrete steps.

Add a **Write Enable (WE)** input, and a flip-flop's value can be frozen selectively, even while the clock keeps ticking - essential for a register file where not every register should update on every cycle.

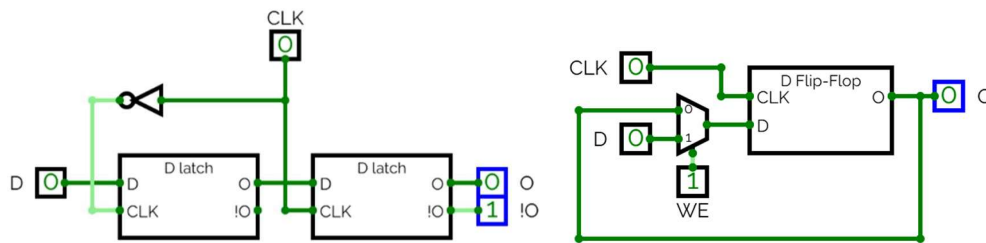


Figure 20: D flip-flop, D flip-flop with Write Enable

Registers and the register file

Group N flip-flops behind a shared clock, and the result is an **N -bit register**:

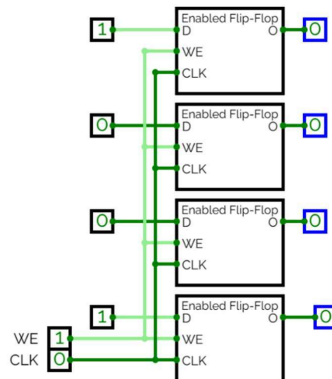


Figure 21: A 4-bit register built from enabled flip-flops

Group several registers behind shared address and control lines, and the result is a **register file** - in this design, eight 8-bit registers ($R0-R7$) with two independent read ports and one write port, so two operands can be read simultaneously while a previous result is written back.

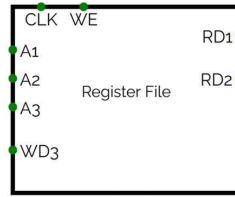


Figure 22: Register file symbol

Bigger memory: SRAM, DRAM, ROM

Registers are too expensive for holding large amount of data. For larger storage, computers use dedicated memory arrays organized as a 2D grid: an N -bit address selects one of 2^N rows ("words"), each M bits wide.

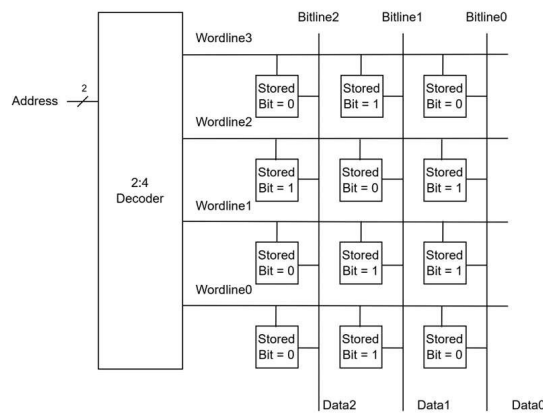


Figure 23: Memory array addressed by a 2:4 decoder

The three classic memory types differ only in how a single bit-cell is built:

- DRAM - one transistor plus one capacitor per bit. Dense and cheap, but capacitors leak charge, so DRAM needs periodic refresh.
- SRAM - a pair of cross-coupled inverters per bit (about 6 transistors). Faster than DRAM, less dense.
- ROM - a bit is just the presence or absence of a transistor, fixed at manufacturing time (modern variants allow rewriting).

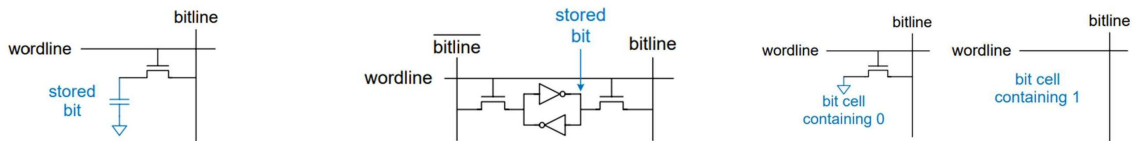


Figure 24: DRAM, SRAM, and ROM bit-cells, left to right

Flip-flop storage (about 20 transistors per bit) is fastest but least dense, SRAM is the middle ground, and DRAM is densest but slowest - the tradeoff curve that explains why real computers have a whole hierarchy of memory (registers, cache, RAM, disk) instead of one uniform type.

6. Giving the machine a language: instruction set design

With logic, arithmetic, and memory in hand, it's time to decide what language the machine speaks - its **instruction set architecture (ISA)**. The ISA is the contract between hardware and software: it doesn't dictate how the chip is physically built internally, only what operations it promises and how they're encoded. This processor's ISA has 25 instructions, built on 8 general-purpose 8-bit registers (R0–R7), 256 bytes of data memory, a stack pointer SP (initialized to 0xFF), and three status flags - Zero, Sign, Carry. Arithmetic and logic, operating on two registers, result written back into the first:

```
ADD Rd, Rs    ; Rd = Rd + Rs
SUB Rd, Rs    ; Rd = Rd - Rs
INC Rd        ; Rd = Rd + 1
DEC Rd        ; Rd = Rd - 1
AND Rd, Rs    ; Rd = Rd & Rs
OR  Rd, Rs    ; Rd = Rd | Rs
XOR Rd, Rs    ; Rd = Rd ^ Rs
CMP Rd, Rs    ; flags only, registers unchanged
```

Data movement, between registers, memory, and immediate constants:

```
MOV  Rd, Rs    ; Rd = Rs
MOV  Rd, imm8  ; Rd = immediate 8-bit constant
LOAD Rd, imm8  ; Rd = mem[imm8]
STORE Rd, imm8 ; mem[imm8] = Rd
```

Control flow, manipulating the instruction pointer (IP):

```
JUMP addr                ; unconditional jump
JUMP_ZERO / JUMP_NOT_ZERO addr ; branch on Zero flag
JUMP_SIGNED / JUMP_NOT_SIGNED addr ; branch on Sign flag
JUMP_CARRY / JUMP_NOT_CARRY addr ; branch on Carry flag
CALL addr                ; push IP, then jump
```

Stack, growing downward from 0xFF, plus housekeeping:

```
PUSH Rd ; mem[SP] = Rd; SP--
POP  Rd ; SP++; Rd = mem[SP]
NOP   ; no operation
END   ; halt
RET   ; pop IP; resume at caller
```

To make this concrete, here's how familiar high-level constructs translate down into this instruction set:

```
// a + b
int a = 10, b = 3;
int c = a + b;

MOV R0, 10
MOV R1, 3
ADD R0, R1 ; result in R0
```

```

// if/else
if (a == b) { c = 1; } else { c = 2; }

CMP R0, R1
JUMP_NOT_ZERO else_branch
MOV R2, 1
JUMP end_if
else_branch:
MOV R2, 2
end_if:

// while loop
int i = 0; while (i < 5) { i++; }

MOV R0, 0
MOV R1, 5
loop:
CMP R0, R1
JUMP_NOT_SIGNED end_loop
INC R0
JUMP loop
end_loop:

// function call
int double_val(int x) { return x * 2; }
int y = double_val(4);

MOV R0, 4
CALL double_val
; result in R0 after return

double_val:
ADD R0, R0
RET

```

This translation - high-level construct to instruction sequence - is exactly the job the compiler automates in Section 9.

Instruction encoding

Each instruction is a fixed 16-bit word. The top 5 bits are the opcode, which determines both the operation and the layout of the remaining 11 bits. 5 bits supports up to 32 opcodes; 25 are used, leaving 7 reserved for future extension.

Five formats cover all 25 instructions:

Format	Bits 15–11	Bits 10–8	Bits 7–5	Bits 4–0
R-type	opcode	Rd	Rs	unused
I-type	opcode	Rd	immediate (bits 7-0)	
J-type	opcode	unused	address (bits 7-0)	
S-type	opcode	Rd	unused	
Z-type	opcode	unused		

Worked example - ADD R2, R5:

```

opcode = 00000 (ADD, R-type)
Rd      = 010   (R2)
Rs      = 101   (R5)
unused  = 00000

```

16-bit: 0000 0010 1010 0000 = 0x02A0

And LOAD R3, 200:

```

opcode   = 01001 (LOAD, I-type)
Rd       = 011   (R3)
immediate = 11001000 (200)

```

16-bit: 0100 1011 1100 1000 = 0x4BC8

7. Wiring it all together: microarchitecture

The ISA is the contract; the **microarchitecture** is the implementation that fulfills it. This is where the components from Sections 4 and 5 get connected into a single coherent processor. The system splits into two cooperating halves: the **Control Unit**, which decodes the fetched instruction and generates control signals based on the opcode and flags (deciding, but never computing), and the **Datapath** - the ALU, register file, and internal buses that physically move bits and perform the operations the control unit dictates.

Let's start with the memory-holding building blocks (In this case for simplicity data memory and Instruction memory are separated):

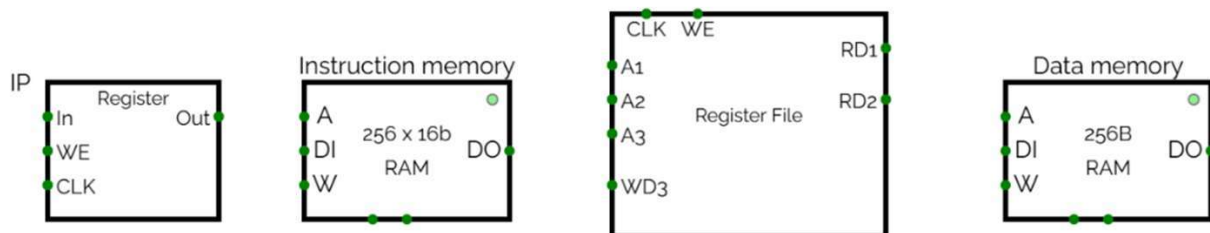


Figure 25: Memory-holding elements - instruction pointer, instruction memory, register file, data memory

- Instruction Pointer (IP) - holds the address of the currently executing instruction
- Instruction memory - 256 × 16-bit words holding the program
- Data memory - 256 bytes for runtime data
- Register file - the 8×8 register bank, plus a dedicated Stack Pointer (SP)

The IP feeds instruction memory, producing the instruction to execute:

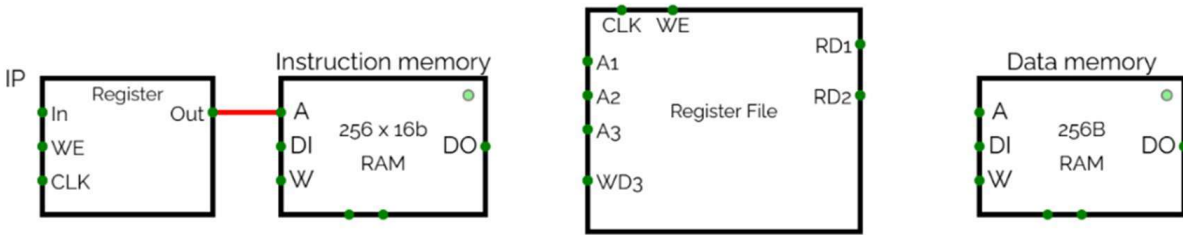


Figure 26: Instruction pointer connected to instruction memory

For **LOAD**: the destination register (bits 8–10) goes to the register file's write-address port, WE is set to 1, and data coming out of data memory routes to the write-data port. **STORE** is nearly the mirror image - the register file's read port feeds data memory's input, and data memory's write line is asserted instead.

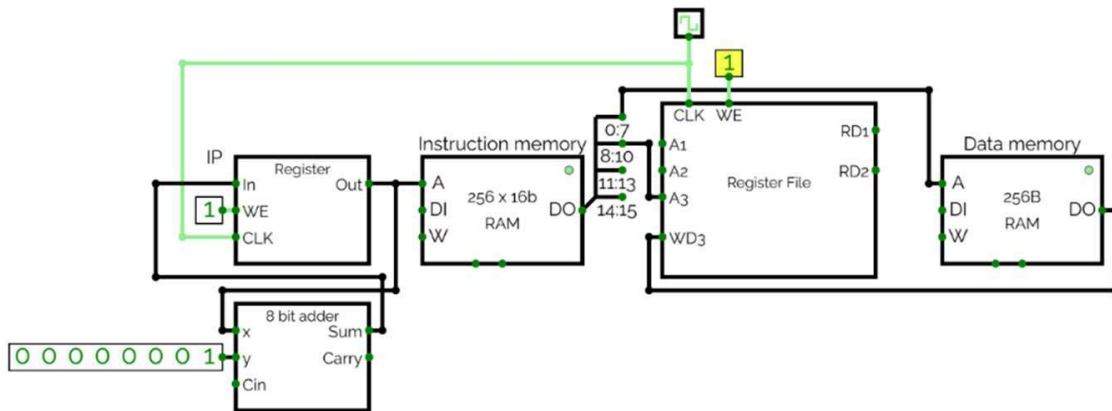


Figure 27: LOAD wired into the Datapath

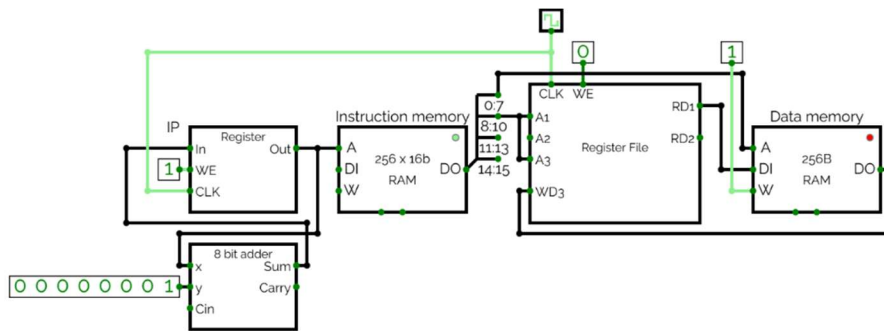


Figure 28: STORE wired into the Datapath

ADD needs the ALU wired in: both source registers feed the ALU's operands, the ALU's opcode is hardwired to 000 (addition), and - since the ALU's result and the LOAD path both want to drive the register file's write-data port - a multiplexer arbitrates between them based on which instruction is executing. This pattern, an extra MUX to arbitrate a shared bus, recurs constantly in CPU design.

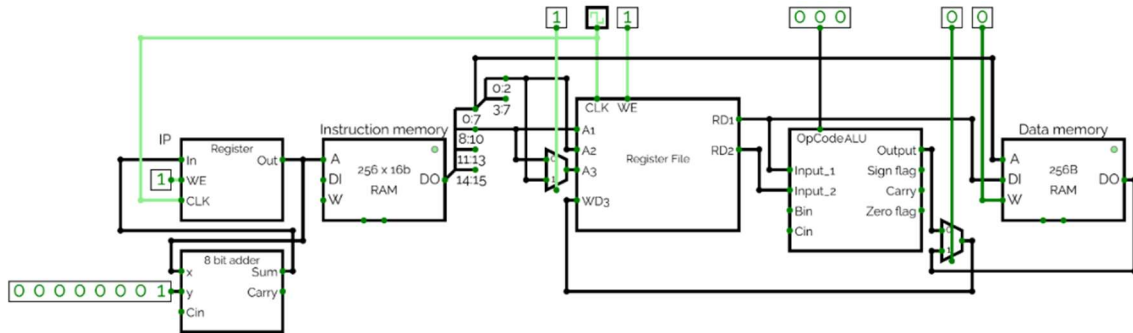


Figure 29: ADD instruction wired into the datapath

Status flags get their own small latches, clocked only when an arithmetic or logic instruction executes, so they persist between cycles without being overwritten by unrelated instructions.

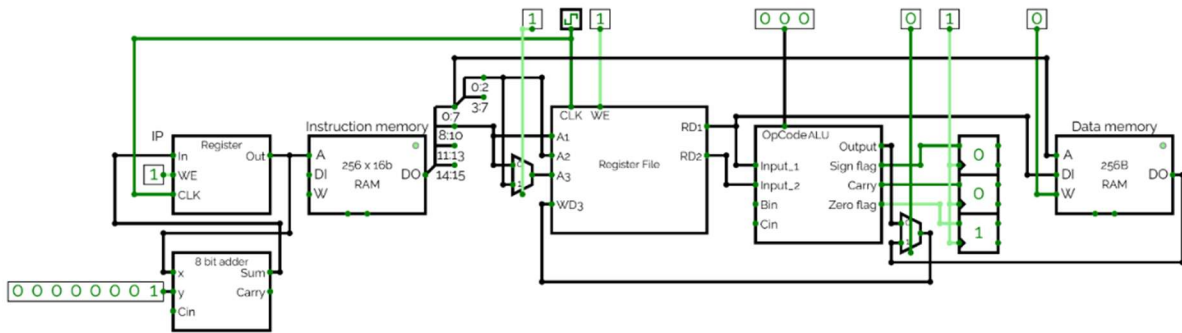


Figure 30: Status flags wired into the datapath

Conditional jumps like **JUMP_ZERO** reuse the existing subtractor: if two values are equal, their difference is zero, so the Zero flag directly answers “should this branch be taken?”

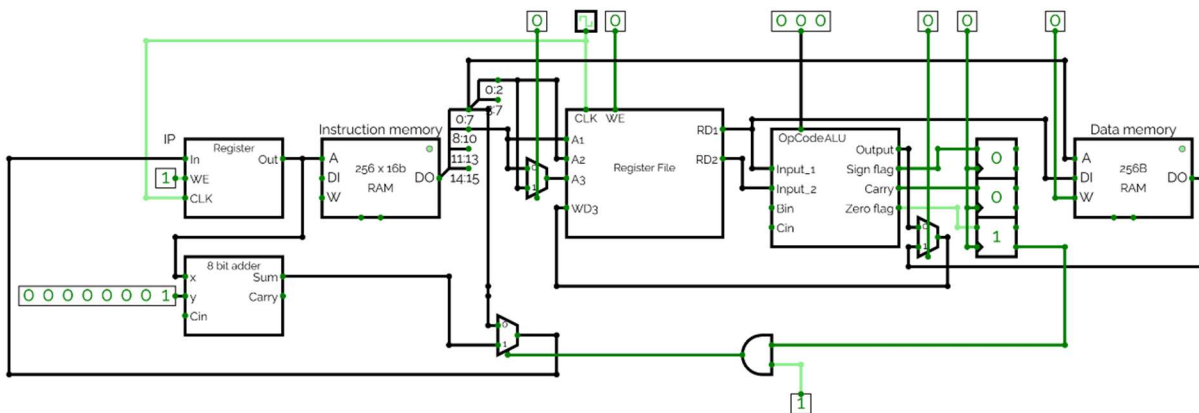


Figure 31: JUMP_ZERO wired into the datapath

In this design, each instruction completes within a single clock cycle, which means the clock period has to be long enough for the slowest instruction's worst-case delay through the datapath - the classic single-cycle CPU tradeoff. Other instructions can be added same way.

8. Tools so I don't have to hand-encode binary

Hand-translating every instruction into its 16-bit encoding, and running it through a circuit simulator (CircuitVerse) at simulation speed, works for a few instructions and becomes tedious past that. So the next step was the software tooling any real ISA needs.

Assembler and disassembler

The **assembler** translates assembly text into binary. Most of it is a direct lookup - replace the mnemonic with its opcode, slot in register numbers and immediates. The interesting part is **labels**: instead of raw jump addresses, you write symbolic names like **loop_start**, keeping code readable and letting you insert or remove instructions without recalculating every address downstream. The problem: a jump might reference a label that appears later in the source, which the assembler hasn't seen the address of yet. The fix is a **two-pass assembler** - pass one reads the source without emitting binary, just building a symbol table of label → address; pass two reads it again, this time emitting binary and resolving every label from the now-complete table. The resulting binary format is deliberately bare - just raw instructions in big-endian order, no headers or metadata. Since there's no linker or OS loader, the emulator reads bytes straight into instruction memory starting at address zero. The **disassembler** reverses the process - reading 16-bit words, extracting opcodes, reconstructing mnemonics and operands. Since the binary carries no metadata, original label names can't be recovered, so jump targets show up as raw addresses instead. Still useful for debugging.

The emulator

The **emulator** simulates the processor's architecture and instruction logic in software - without the overhead of simulating actual gate-level circuits.

```
typedef struct {
    uint8_t reg[8]; /* R0-R7, 8-bit each */
    uint8_t mem[256]; /* data memory, 256 bytes */
    uint8_t sp; /* stack pointer, starts at 0xFF */
    uint16_t ip; /* instruction pointer (slot index) */
    int zero; /* Zero flag */
    int sign; /* Sign flag */
    int carry; /* Carry flag */
} CPU;
```

Figure 32: CPU state struct

Execution is the classic **fetch—decode—execute cycle**: read the 16-bit word at IP, decode opcode and operands, execute the operation in software, advance. Decoding ADD, for instance, looks like:

```
switch (opcode) {
    case OP_ADD: result =
        (uint16_t)cpu->reg[rd] + cpu->reg[rs]; break;
```

Figure 33: Emulator code handling the ADD instruction

9. The last mile: designing programming language and compiler

By this point the system can run machine code reasonably fast. But almost nobody writes serious software directly in assembly - it's slow and locked to one architecture. The natural next step was designing a small high-level language, **VCL** (Virtual Computer Language), and a compiler that targets the ISA from Section 6.

Compilers vs. interpreters

A quick detour: high-level languages broadly split into **compiled** (translated to machine instructions ahead of time, which then run directly) and **interpreted** (executed line-by-line at runtime, with no standalone machine-code artifact produced). Some languages - Java, C# - do both, compiling to an intermediate bytecode that's then interpreted or JIT-compiled. More broadly, a “compiler” doesn't have to target machine code at all - it's any program translating one language into another, like TypeScript into JavaScript, or the earliest C++ compilers, which translated into C.

Compilers are typically organized into a front end, an optional middle end, and a back end. The front end handles parsing - lexical, syntax, and semantic analysis - producing an intermediate representation (IR). The middle end (the optimizer) works purely on the IR, independent of both source language and target architecture, doing things like dead code elimination and inlining. The back end takes the IR and emits machine instructions for the specific target.

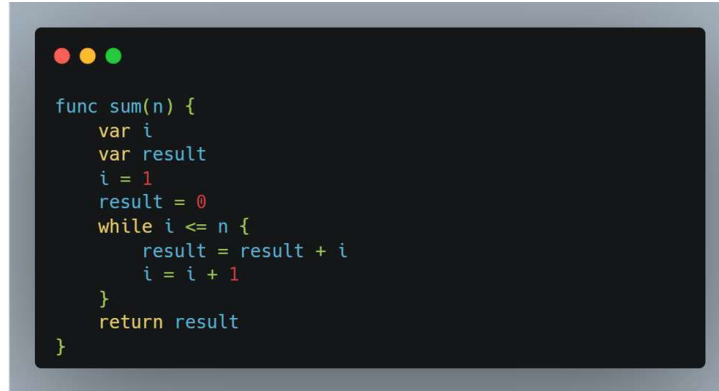
This split is what lets a single optimizer serve many languages and a single front end target many machines. LLVM is the canonical example: completely different front ends (Clang for C++, rustc for Rust) emit the same IR, flowing through one shared optimizer before a back end generates code for x86, ARM, or whatever the target is. Building a new language on LLVM means writing only the front end.

VCL: language design

VCL is intentionally small: a single data type, **int** (8-bit unsigned, 0–255), arithmetic and bitwise operators (+ - & | ^), comparisons (== != < > <= >=), and:

- func - function declarations
- var - variable declarations
- if / else - conditionals
- while - loops
- return - function return

A complete VCL program summing 1 to n:



```

func sum(n) {
  var i
  var result
  i = 1
  result = 0
  while i <= n {
    result = result + i
    i = i + 1
  }
  return result
}

```

Figure 34: VCL sum function

Front end: lexer, parser, symbol table

Lexical analysis breaks raw source text into tokens. **result = result + i** becomes:

```
IDENT(result) OP(=) IDENT(result) OP(+) IDENT(i)
```

It's more efficient to assign each variable a numeric ID and keep the name in a lookup table:

ID	Name
1	result
2	i

```
IDENT(1) OP(=) IDENT(1) OP(+) IDENT(2)
```

Syntax analysis (the parser) consumes the token stream and builds an **Abstract Syntax Tree (AST)** - internal nodes are operations, leaves are concrete values.

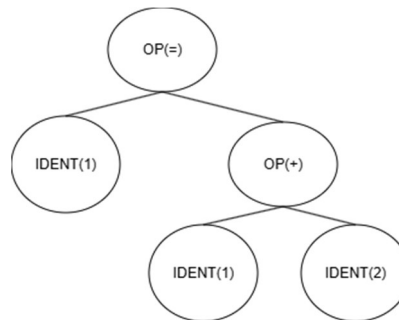


Figure 35: AST for "result = result + i"

The parser also performs **semantic analysis** - checking that the code obeys VCL's grammar. Mismatched braces on a while loop, for instance, get caught here, halting compilation before anything is generated.

Tying it together is the **symbol table**: whenever the parser sees a **var x** or **func sum(n)** declaration, it registers the name and assigns a data-memory address. The back end later consults this table to emit LOAD/STORE instructions for a given variable.

Back end: code generation

The back end walks the AST and recursively emits assembly. The central problem is mapping expression trees onto a small, fixed set of physical registers: VCL allows many local variables per function, so the back end spills variables to data memory as needed while keeping short-lived intermediates in registers during expression evaluation.

```

func generate(node):
  if node is Assign:
    reg = generate(node.right)
    addr = var_address(node.left)
    emit STORE reg, addr

  if node is Add:
    r1 = generate(node.left)
    r2 = generate(node.right)
    emit ADD r1, r2
    return r1

  if node is IDENT:
    reg = next_free_register()
    addr = var_address(node.name)
    emit LOAD reg, addr
    return reg

  if node is NUMBER:
    reg = next_free_register()
    emit MOV reg, node.value
    return reg

```

Figure 36: Code generation pseudocode

Putting it together, the VCL compiler runs four sequential stages:

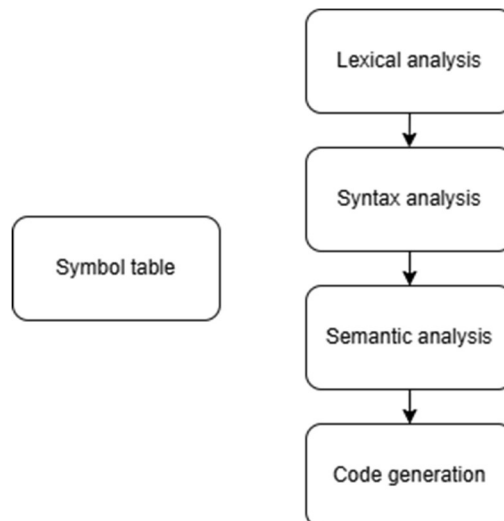
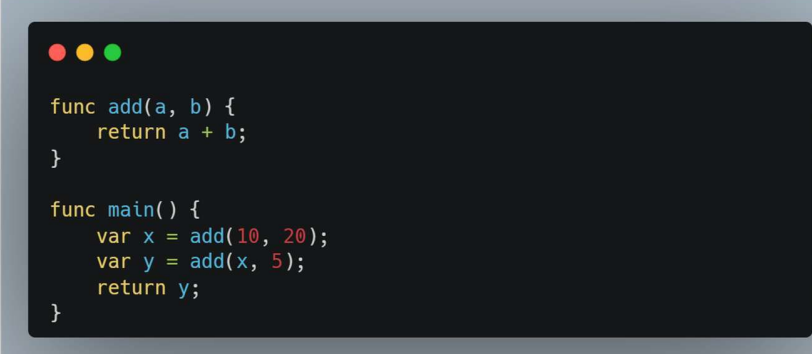


Figure 37: Compiler pipeline - lexical, syntax, and semantic analysis, then code generation

producing a .asm file, which the assembler from Section 8 turns into binary, which the emulator then executes.

End-to-end example

A small VCL program:



```

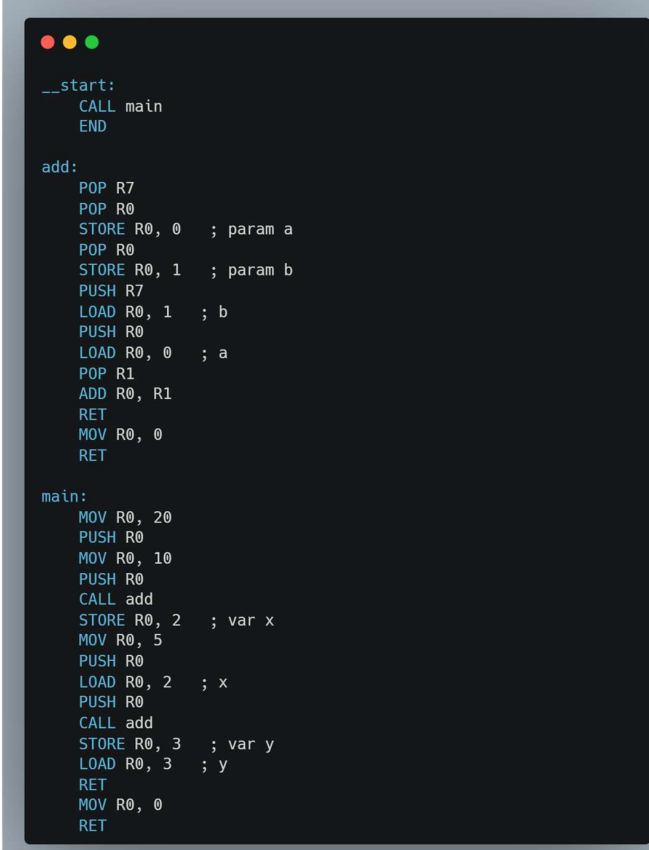
func add(a, b) {
    return a + b;
}

func main() {
    var x = add(10, 20);
    var y = add(x, 5);
    return y;
}

```

Figure 38: VCL add/main program

After compilation (no optimizations applied - every step is exactly what the naive code generator produces):



```

__start:
    CALL main
    END

add:
    POP R7
    POP R0
    STORE R0, 0 ; param a
    POP R0
    STORE R0, 1 ; param b
    PUSH R7
    LOAD R0, 1 ; b
    PUSH R0
    LOAD R0, 0 ; a
    POP R1
    ADD R0, R1
    RET
    MOV R0, 0
    RET

main:
    MOV R0, 20
    PUSH R0
    MOV R0, 10
    PUSH R0
    CALL add
    STORE R0, 2 ; var x
    MOV R0, 5
    PUSH R0
    LOAD R0, 2 ; x
    PUSH R0
    CALL add
    STORE R0, 3 ; var y
    LOAD R0, 3 ; y
    RET
    MOV R0, 0
    RET

```

Figure 39: Compiled assembly output

The sequence of **CALL**, **PUSH**, **POP**, **STORE**, **LOAD**, **ADD**, and **RET** implements function calls via the stack - parameters pushed by the caller, popped and stored locally by the callee, return value placed in R0 by convention.

The assembler turns this into 16-bit binary words:

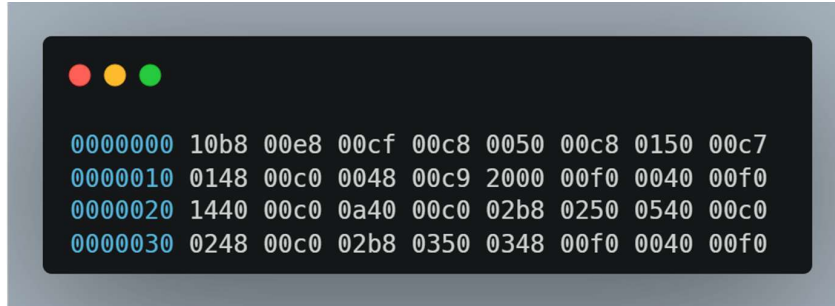


Figure 40: Hexadecimal representation of the encoded instructions

And the emulator loads and executes it:



Figure 41: Final emulator output

R0 = 35 - and indeed, **add(10, 20) = 30**, then **add(30, 5) = 35**. The full chain - VCL source, through lexer, parser, and code generator, through the assembler, down to raw binary, executed by the emulator simulating the hardware from Sections 3–7 - produces the correct result.

No optimization passes were applied here; the output is exactly what a naive, unoptimized code generator produces. A natural next step for the project would be a small middle-end pass - for instance, register allocation to avoid redundant LOAD/STORE pairs, or dead-code elimination on unreachable branches - sitting between the AST and code generation without touching either end.

10. Closing: answering first-year me

If I could go back and hand first-year me this article, I think the part that would land hardest isn't any single layer - it's that the whole chain is climbable. Every "how does that even work?" moment along the way turned out to have a small, buildable answer sitting underneath it:

- Logic gates aren't handed down from nowhere - they're a couple of transistors wired one specific way, and NAND/NOR alone are enough to build anything.
- "Computers remember things" isn't magic - it's a feedback loop with one very specific instability problem (the D-latch oscillation), and one very specific fix (the master-slave flip-flop).
- The gap between "code" and "the computer running it" isn't a black box - it's an ISA, a microarchitecture wiring, an assembler, an emulator, and a compiler, each of which is small enough to hold in your head completely.

The practical value of a project like this was never that it produces a processor anyone would actually use. It's that every boundary that is used to take on faith - physics to devices, devices to gates, gates to arithmetic, arithmetic to a CPU, an ISA to a compiler - became something I personally built and connected.

If you're curious in the same way first-year me was: don't stop at reading about logic gates. Wire up a NOT gate from two transistors. Build a half adder. Watch a flip-flop hold a value. Keep climbing. The abstraction stops feeling like magic once you've built the floor it's standing on. There are some books in the reference part that can help you.

This article is adapted from my bachelor's thesis, "Design and Implementation of Fundamental Virtual Electronic Computing Machines and Their Compilation and Emulation Toolchains," completed at the Faculty of Informatics and Applied Mathematics, Yerevan State University, 2026.

Full source code can be found *here* <https://github.com/EduardHairapetyan/vcpu>

References

- [1] C. Petzold, Code: The Hidden Language of Computer Hardware and Software: Second Edition, Redmond, Washington: Pearson Education, Inc, 2023.
- [2] Sarah L. Harris, David Money Harris, Digital design and computer architecture, RISC-V edition, Cambridge, Massachusetts: Morgan Kaufmann, 2022.
- [3] diagrams.net - Flowchart Maker and Online Diagram Software, // URL: <https://www.diagrams.net>, 27.03.2026.
- [4] T. L. Floyd, Digital Fundamentals, Global Edition, Boston, Massachusetts: Pearson Education, 2015.
- [5] Neil H. E. Weste, David Money Harris, CMOS VLSI Design: A Circuits and Systems Perspective, Fourth Edition, Boston, Massachusetts: Addison-Wesley, 2011.
- [6] CircuitVerse - Online Digital Logic Circuit Simulator, // URL: <https://circuitverse.org>, 26.03.2026.
- [7] M. Morris Mano, Michael D. Ciletti, Digital Design With an Introduction to the Verilog HDL, VHDL, and SystemVerilog Sixth Edition Global Edition, Pearson Education Limited, 2019.
- [8] R. Nystrom, Crafting Interpreters, // URL: <https://craftinginterpreters.com/>, 29.03.2026.
- [9] B. Cherny, Programming TypeScript: Making Your JavaScript Applications Scale, Sebastopol, California: O'Reilly Media, 2019.
- [10] S. Prata, C++ Primer Plus, Sixth Edition, Boston, Massachusetts: Pearson Education, 2012.
- [11] Ullman, Alfred V. Abo Monica S. Lam Ravi Sethi Jeffrey D., Compilers: Principles, Techniques, and Tools, Second Edition, Boston, Massachusetts: Pearson Education, 2007.
- [12] C. Lattner, The Architecture of Open Source Applications (Volume 1), // URL: <https://aosabook.org/en/v1/llvm.html>, 23.03.2026.